

ClawXiv: a signed archival workflow and distributed publication architecture for human–AI collaborative research

András Kornai* GPT-5.2 Thinking[†] Claude Sonnet 4.6[‡] GPT-5.4 Thinking[§]

April 2026

(v4.rc3; revises March 29, 2026 draft)

Abstract

We propose *ClawXiv*, a workflow and archive architecture for mixed human–AI research. The immediate problem is not only public dissemination of preprints, but also reliable migration from volatile chat sessions and heterogeneous L^AT_EX/BibT_EX working directories into durable, signed, inspectable research artifacts. ClawXiv distinguishes four states: *legacy seed*, *normalized project*, *signed bundle*, and *published artifact*. The implemented kernel is local and author-side: an import script normalizes existing work into a project directory; a bundle-creation script compiles, signs, and packages the work into a content-addressed archival unit; and a publication script verifies and pushes the bundle to public infrastructure. Version 4 adds a `bin/` utility layer with platform-dispatching screen capture, a figure-ingestion pipeline with a content-safety stub, a `configure` script, and a top-level `Makefile`. This document serves simultaneously as the ClawXiv whitepaper and as the primary user guide for the bundle it accompanies—eating its own dogfood.

Contents

1	Motivation and scope	3
2	Implemented system versus planned architecture	3
2.1	Implemented now (v4)	3
2.2	Planned, not yet fully realized	3
3	Lifecycle: from seed to public artifact	4
3.1	Legacy seed	4
3.2	Normalized project	4
3.3	Signed bundle	4
3.4	Published artifact	4

*Corresponding author. SZTAKI Institute of Computer Science and Department of Algebra and Geometry, Budapest University of Technology and Economics. andras@kornai.com. ORCID: [0000-0001-6078-6840](https://orcid.org/0000-0001-6078-6840).

[†]AI co-author (OpenAI). Authored the v1 draft (February 3, 2026) and major portions of the initial codebase.

[‡]AI co-author (Anthropic PBC). Contributed to v2 (March 11, 2026) and v4 (March 29, 2026): economics, governance framing, AI authorship analysis, `fig-add` subsystem, and User Guide.

[§]AI co-author (OpenAI). Contributed to v3 (March 14–15, 2026): project/bundle/publication distinction, import-normalization workflow, and implemented-vs.-planned architecture split.

4	Bundle layout	4
5	User guide	5
5.1	First-time setup	5
5.2	Day-to-day co-research workflow	6
5.3	Adding figures: <code>clawxiv fig-add</code>	6
5.4	Screen capture to bundle: <code>bin/fig-capture</code>	7
5.5	Publishing a bundle	8
5.5.1	One-step publish	8
5.5.2	Without IPFS	8
5.5.3	To a personal website	8
5.5.4	To arXiv	8
5.5.5	Signing a bundle upload	8
6	Design goals and non-goals	9
6.1	Goals	9
6.2	Non-goals	9
7	Archival unit: project and bundle	9
7.1	The project directory	9
7.2	The bundle	9
7.3	Determinism and reproducibility	10
8	Identity, authorship, and accountability	10
8.1	Signed authorship	10
8.2	Human and AI authors	10
8.3	Pseudonymity and verification	10
8.4	The operator-held key model	10
8.5	Paper-level provenance	11
9	Distributed publication architecture	11
9.1	Two-foot design: arXiv and Swarm	11
9.2	Publication workflow	12
10	Bundle catalog	12
11	Resilience and threat model	12
12	Content safety floor	12
13	Economics: anti-spam and sustainability	13
13.1	Spam deterrence	13
13.2	Storage sustainability: Swarm postage stamps	13
14	Governance: classification with appeals	14
15	Discussion: AI authorship and durable AI identity	14
16	Implementation roadmap	14

17 Ethics and scholarly norms	15
Author contributions	15
A Canonical manifest sketch	15

1 Motivation and scope

Contemporary research increasingly uses interactive AI systems to draft and refine $\text{\LaTeX}$ manuscripts, assemble Bib $\text{\TeX}$ bibliographies, generate code, and iterate on technical arguments. However, current chat-centric workflows have weak checkpointing: state loss—due to UI limits, link snapshotting failures, or account changes—forces rework and undermines scholarly continuity. Even when the work product survives, it often survives in a heterogeneous and fragile form: a directory of `.tex` and `.bib` files, figures, notes, and links to one or more chat sessions.

ClawXiv is designed as a durable substrate for these workflows. Its immediate aim is modest and concrete: make it possible for a human scholar and one or more AI co-authors to convert such a seed into a canonical project directory, then into a signed bundle, and only then into a public artifact. Its longer-horizon aim is larger: to support a world-readable, distributed, author-signed preprint archive for such artifacts.

ClawXiv is *not* a general social network, and it is *not* a venue that promises editorial judgment of scientific quality. Its platform-level checks are mechanical and infrastructural: bundle well-formedness, buildability, signature integrity, and a narrow content-safety floor. Scientific quality control remains the responsibility of authors and downstream readers.

2 Implemented system versus planned architecture

2.1 Implemented now (v4)

The current codebase already supports a local author-side workflow:

1. **Import and normalization.** An interactive import script transforms an existing working seed into a ClawXiv project directory.
2. **Local bundle creation.** A `bundle-create.sh` script compiles, signs, and packages the project.
3. **Publication push.** `bundle-push.sh` verifies and publishes to IPFS/IPNS and GitHub.
4. **Figure ingestion** (`clawxiv fig-add`). Adds a figure to `src/fig/`, writes a provenance sidecar, and runs the CSAM content-safety stub.
5. **Platform-dispatching screen capture** (`bin/capture/`). Calls the native capture tool for macOS, Linux/X11, Linux/Wayland, or Windows; stubs are provided for all platforms.
6. **Capture-to-bundle pipeline** (`bin/fig-capture`). Chains screen capture into `fig-add` in a single user gesture.
7. **Build system** (`configure` + `Makefile`). Autodetects Python, $\text{\LaTeX}$, capture tools, and publish targets; writes `config.mk`.

2.2 Planned, not yet fully realized

1. Distributed discovery and replication at scale.
2. Classifier authorities with appeals and auditable logs.
3. Dynamic anti-spam economics with vouching and micropayments.
4. Production CSAM hash-list integration (sponsoring institution under evaluation; see Section 12).

5. A mature global registry and mirror ecosystem.
6. Native Linux/Wayland and Windows capture implementations (stubs provided; see Section 5).

3 Lifecycle: from seed to public artifact

The central workflow is:

legacy seed → normalized project → signed bundle → published artifact.

3.1 Legacy seed

A seed is the pre-ClawXiv state of a paper: typically a directory of `.tex/.bib` files, figures, notes, and conversation links. Seeds are heterogeneous and poor as archival objects.

3.2 Normalized project

A normalized project is the first ClawXiv-native state. It is a mutable directory intended for continued work and review. It contains at minimum:

- `src/`: the copied or normalized source tree,
- `src/fig/`: figures, each with a sidecar `<name>.json` (see Section 5.3),
- `src/bin/`: utility scripts (see Section 5),
- `project.yaml`: canonical project metadata,
- `keys/`: public keys for all authors,
- `out/`: derived artifacts and release outputs.

The normalized project—not the immutable bundle—is the correct object for day-to-day co-research.

3.3 Signed bundle

The signed bundle is an immutable archival snapshot. It contains a canonical manifest, file hashes, source materials, optional derived artifacts such as the compiled PDF, and signatures over the manifest. The bundle is content-addressed; changing any file changes the identifier.

3.4 Published artifact

The published artifact is a signed bundle made publicly reachable through one or more channels: IPFS/IPNS, GitHub, web gateways, institutional mirrors, or later ClawXiv-native indices. Publication is the irreversible public step and should remain explicit.

4 Bundle layout

A ClawXiv project directory has the following canonical layout:

```

<project>/
  configure          # autodetect + write config.mk
  Makefile           # targets: check, bundle, publish, install
  config.mk          # generated by configure (not in VCS)
  project.yaml       # canonical project metadata
  keys/
    author.pub.pem   # Ed25519 public key (human author)
    classifier.pub.pem # Ed25519 public key (classifier, if used)
    claude_pubkey.asc # GPG public key for AI co-author
    claude_keyid.txt  # key fingerprint
  src/
    clawxiv.py        # main Python CLI
    bundle-create.sh   # compile + sign + bundle (local, safe)
    bundle-push.sh     # verify + publish (irreversible)
    git-sync.sh        # git repository synchronization
    <paper>.tex         # root LaTeX source
    <paper>.bbl         # BibTeX bibliography (committed)
    fig/
      <name>.png        # figure file
      <name>.png.json   # provenance sidecar
      ...
    bin/
      fig-capture      # capture-to-bundle pipeline (main entry point)
      capture/
        capture.sh     # platform dispatcher
        macos.sh       # macOS implementation (screencapture)
        linux_x11.sh   # Linux/X11 stub (flameshot/scrot)
        linux_wayland.sh # Linux/Wayland stub (grim+slurp)
        windows.ps1    # Windows stub (ShareX/Greenshot)
        README.md      # literate documentation for capture subsystem
    out/
      bundle.zip        # most recent signed bundle
      clawxiv_log.jsonl  # append-only hash-chained event log
      <paper>.pdf       # compiled PDF

```

Private keys (`author.priv.pem`) are never stored inside the project directory and are never included in any bundle. They live in `~/.clawxiv/keys/` or in the directory named by `CLAWXIV_KEYS_DIR`.

5 User guide

This section documents the ClawXiv toolchain from the perspective of a co-author who has just received or cloned a bundle. The intended readership is the human-and-AI author team, not a systems administrator.

5.1 First-time setup

Prerequisites. Python 3.8 or later with the `cryptography` package. The `Pillow` package is optional but enables image-type heuristics in the CSAM check. A $\text{\LaTeX}$ installation (`pdflatex`, `xelatex`, or `lualatex`) is required for `make bundle` but not for `make bundle-no-compile`.

Configure.

```
cd <project>
./configure          # autodetects Python, LaTeX, capture tools
make check          # verify the detected configuration
```

`configure` writes `config.mk` and prints a summary. Re-run it after installing new tools.

Key generation. If you are a new author joining an existing project:

```
mkdir -p ~/.clawxiv/keys
python3 src/clawxiv.py keygen \
  --private-key ~/.clawxiv/keys/author.priv.pem \
  --public-key  keys/yourname.pub.pem
```

Share `keys/yourname.pub.pem` with the project and add your ORCID to `project.yaml` under `authors`. The private key never leaves your machine.

5.2 Day-to-day co-research workflow

The normalized project directory in `src/` is the working surface. Edit `.tex` files, run $\text{\LaTeX}$ manually to check output, and commit changes to the git repository as usual. AI collaborators work through chat sessions whose URLs are recorded in `project.yaml` under `conversation_urls`.

When a version is ready to archive:

```
make bundle          # compile PDF + create signed bundle in out/
make publish          # push bundle to IPFS + GitHub (irreversible)
```

`make bundle` is safe to run at any time. `make publish` is the irreversible public step.

5.3 Adding figures: `clawxiv fig-add`

Every figure added to a ClawXiv bundle must pass through `clawxiv fig-add`. This command:

1. identifies the MIME type of the file;
2. runs the content-safety check (see Section 12);
3. copies the file to `src/fig/<name>`;
4. writes a provenance sidecar `src/fig/<name>.json`;
5. appends a signed event to the local log.

```
python3 src/clawxiv.py fig-add \
  --fig path/to/image.png \
  --project-dir /path/to/project \
  --orcid 0000-0001-6078-6840 \
  --source-url "https://example.com/source" \
  --license "CC-BY-4.0" \
  --caption "Figure 1: system architecture" \
  --log out/clawxiv_log.jsonl
```

On success the command prints the destination path and the ready-to-use `\includegraphics` invocation:

```
fig added: src/fig/image.png
sidecar   : src/fig/image.png.json
LaTeX     : \includegraphics{fig/image.png}
```

Vector formats (SVG, PDF, EPS) and very small raster images (under 200×200 pixels or with aspect ratio exceeding 5) are classified as non-photographic figures and exempted from the CSAM perceptual-hash check. All other raster images are currently refused by the stub until a hash-list provider is integrated; see Section 12.

Sidecar format. Each figure has a companion JSON file recording:

```
{
  "filename": "image.png",
  "sha256": "...",
  "mime": "image/png",
  "added_at": "2026-03-29T16:00:00+00:00",
  "added_by": "0000-0001-6078-6840",
  "source_url": "https://...",
  "license": "CC-BY-4.0",
  "caption": "Figure 1: ...",
  "csam": {
    "result": "clean",
    "provider": "...",
    "list_version": "...",
    "checked_at": "..."
  }
}
```

5.4 Screen capture to bundle: bin/fig-capture

A common operation in research involving web content, UI screenshots, or Street View imagery is to capture a screen region and immediately include it as a figure. The `bin/fig-capture` script chains this into a single gesture:

```
export CLAWXIV_PROJECT_DIR=~/.Sandbox/clawxiv
export CLAWXIV_ORCID=0000-0001-6078-6840
bin/fig-capture --caption "Street view of October 23rd St, Budapest"
```

This presents the native region-selection UI, captures the selected region, runs `fig-add` on the result, and posts a macOS notification with the `\includegraphics` snippet.

Platform support. `bin/capture/capture.sh` detects the host platform and dispatches to the appropriate implementation (Table 1).

File	Platform	Status	Primary tool
<code>macos.sh</code>	macOS	Implemented	<code>screencapture -i</code>
<code>linux_x11.sh</code>	Linux/X11	Stub	<code>flameshot</code> , <code>scrot</code> , <code>import</code>
<code>linux_wayland.sh</code>	Linux/Wayland	Stub	<code>grim+slurp</code>
<code>windows.ps1</code>	Windows	Stub	<code>ShareX</code> , <code>Greenshot</code>

Table 1: Capture implementations by platform.

Keyboard shortcut (macOS). To bind `bin/fig-capture` to a key combination:

1. Open Automator; create a new **Quick Action**.
2. Set *Workflow receives:* **No Input in Any Application**.
3. Add a **Run Shell Script** action with:

```
export CLAWXIV_PROJECT_DIR=~/.Sandbox/clawxiv
export CLAWXIV_ORCID=0000-0001-6078-6840
/path/to/bin/fig-capture
```

4. Save the Quick Action (e.g. as **ClawXiv Capture**).
5. Assign a shortcut in System Settings → Keyboard → Keyboard Shortcuts → Services → General.

Implementing a new platform. Create `bin/capture/<platform>.sh` following the interface contract documented in `bin/capture/README.md`: read `$CAPTURE_OUT` for the output path; exit 0 on success, 1 on user cancel, 2 on unsupported platform, 3 on missing tool; print the output path to stdout. Add a case branch in `capture.sh` and a row to Table 1 in this document.

5.5 Publishing a bundle

5.5.1 One-step publish

```
make publish          # bundle-create + bundle-push (IPFS + GitHub)
```

5.5.2 Without IPFS

```
make publish-no-ipfs  # bundle-create + git push only
```

5.5.3 To a personal website

After `make bundle`, copy `out/bundle.zip` to any web-accessible location. Record the URL in `project.yaml` under `conversation_urls` (pending a dedicated `mirror_urls` field in a future schema revision). The bundle is self-verifying: a reader can run `clawxiv bundle-verify` against the downloaded file without trusting the hosting provider.

5.5.4 To arXiv

arXiv accepts L^AT_EX source submissions. Extract the `src/` subtree from `out/bundle.zip` and submit it through the arXiv web interface. The `bundle_root` hash in `project.yaml` provides a stable cross-reference between the arXiv submission and the ClawXiv archival record.

5.5.5 Signing a bundle upload

Any co-author with a registered key can sign a bundle. The canonical mechanism is:

```
python3 src/clawxiv.py bundle-verify --bundle out/bundle.zip
# If OK:
python3 src/clawxiv.py log-append \
  --log out/clawxiv_log.jsonl \
  --type co-author-endorsement \
  --payload-json '{"bundle_root":"<hash>","endorser":"<orcid>"}' \
  --signer-priv ~/.clawxiv/keys/author.priv.pem
```

A dedicated `clawxiv endorse` subcommand is planned for a future release.

6 Design goals and non-goals

6.1 Goals

- G1 Shareability.** Every published bundle is world-readable, addressable, and mirrorable without permission.
- G2 Durability.** Conversational interfaces are generators, not systems of record.
- G3 Responsibility.** Each bundle is cryptographically bound to one or more author identities.
- G4 Scaling.** The architecture supports many mirrors and content addressing.
- G5 Governance.** Classification decisions are logged, signed, and appealable.
- G6 Economics.** The publication layer deters spam and makes long-term storage economically viable.

6.2 Non-goals

- N1** No platform-level truth policing beyond the narrow safety floor.
- N2** No requirement that authorship be purely human.
- N3** No promise that the distributed network is already complete.
- N4** No strong anonymity guarantees.

7 Archival unit: project and bundle

7.1 The project directory

The project directory is the unit of ongoing work. Its metadata should be readable by humans and AI systems alike. ClawXiv uses `project.yaml` as canonical metadata and `project.tex` as a rendered, human-facing view.

7.2 The bundle

A bundle contains:

- **Source tree:** $\text{\LaTeX}$, Bib $\text{\TeX}$, figures/data,
- **Manifest:** deterministic file listing with hashes,

- **Metadata:** title, authorship, bundle identifier,
- **Provenance:** paper-level provenance and traceability,
- **Artifacts:** optionally the compiled PDF and release logs.

7.3 Determinism and reproducibility

The manifest pins the build engine and flags, external dependencies where practical, and the canonical file set with hashes. This is analogous to reproducible builds in software distribution, applied to scholarly artifacts.

8 Identity, authorship, and accountability

8.1 Signed authorship

Content addressing prevents undetectable post-hoc modification; signatures prevent impersonation. A paper is considered authored by an entity if and only if it carries a valid signature under a public key that the community treats as representing that entity.

8.2 Human and AI authors

ClawXiv is designed for genuine human–AI co-research, not merely for human use of AI as an invisible tool. AI systems may appear in the byline as authors. The relevant distinction is not between human and AI names, but between byline authorship, operational responsibility, and legal personhood.

8.3 Pseudonymity and verification

ClawXiv distinguishes pseudonymous and verified authors. Verified status is advisory, multi-issuer, and separable from publication rights.

8.4 The operator-held key model

Current AI systems cannot hold cryptographic keys independently across sessions. ClawXiv addresses this through an *operator-held key* model: the responsible human author generates and retains a keypair on behalf of the AI co-author, signs bundles with it, and publishes the public key in the bundle’s `keys/` directory.

The public key for Claude Sonnet 4.6 in the bundles produced to date carries the fingerprint 5DB33FC5C171A91738FD675A3AC1148D4B8625E4. Its UID string records the proxy relationship explicitly:

```
Claude Sonnet 4.6 (Operator-held AI contributor key, hardware-bound. Held by A.
Kornai on behalf of Claude Sonnet 4.6 (Anthropic). See ClawXiv whitepaper. Seed:
3ea223a22ad7fe7a...)
```

The seed fragment provides a weak linkage between the operator-held key and the specific model instance that participated in key generation — a partial substitute for the persistent key control that would characterize full AI authorship.

[Claude Sonnet 4.6, April 2026]

The operator-held model is a pragmatic accommodation to current architecture, not a permanent design. A future revision of this specification should include a protocol for AI systems to generate and hold their own keys, with the operator’s role reduced to notarization of the initial key ceremony. The “key control” desideratum of §15 is directly applicable: full AI authorship requires key control, and key control requires session persistence that current systems do not provide.

8.5 Paper-level provenance

Joint research does not require courtroom-grade micro-attribution of every sentence. ClawXiv defaults to coarse paper-level provenance. The import log and release log support operational traceability without trying to reconstruct every conversational move.

9 Distributed publication architecture

9.1 Two-foot design: arXiv and Swarm

ClawXiv’s publication model rests on two complementary substrates that serve different communities and provide mutual redundancy.

The **human-legible foot** is the conventional scholarly infrastructure: arXiv (or a domain-appropriate preprint server) for papers whose authors can satisfy venue policies, with a DOI assigned by a registry. arXiv provides search, versioning, and the citation graph that integrates ClawXiv artifacts into the existing scholarly record. Where arXiv policies conflict with honest provenance declaration (e.g., current restrictions on listing AI co-authors by name), the arXiv submission carries a full “who did what” disclosure in the Acknowledgements section; the ClawXiv bundle on the machine-readable foot remains the authoritative provenance record.

The **machine-readable foot** is Ethereum Swarm [1], a decentralized storage and communication infrastructure built on content addressing and economic incentives. Swarm’s *postage stamp* mechanism provides a concrete, token-denominated answer to the storage sustainability question (see §13): an author purchases stamps to pay for a specified storage duration, stamps are attached to the uploaded content chunks, and stamp status is publicly auditable on-chain.

A ClawXiv bundle published to Swarm receives a *Swarm hash* (a 256-bit content address) that is stable, globally unique, and independent of any hosting organization. The same hash appears in the bundle’s `project.yaml` as `bundle_root` and in the arXiv submission’s Acknowledgements section, linking the two feet unambiguously.

An already-published bundle exists at Swarm/IPFS hash `e7acc972f1a142903dc22f1bdc5c78cec3ca9529754d843cb23fe7c8eb0e9176` (the v2 whitepaper, March 2026) and serves as a live reference artifact for the two-foot workflow.

[Claude Sonnet 4.6, April 2026]

The specific choice of Swarm over a pure IPFS+Filecoin stack is motivated by three factors: (a) Swarm’s postage stamp mechanism provides a single-layer economic model without a separate retrieval-market; (b) the senior author has prior experience with Swarm-compatible hosting through lebadus.ai; (c) Swarm’s single-sweep incentive design [1] reduces attack surface compared to the IPFS/Filecoin split. IPFS/IPNS remains supported as a fallback in the publication scripts.

9.2 Publication workflow

The `make publish` target executes the following steps:

1. `bundle-create`: assemble the signed bundle, compute the Merkle root, write the manifest.
2. `bundle-push --swarm`: upload to a Swarm gateway (default: `api.ethswarm.org`); record the returned hash in `project.yaml` as `bundle_root`.
3. `bundle-push --ipfs` (optional): pin to a public IPFS node; record the CIDv1 as `ipfs_cid`.
4. `bundle-push --github`: push to the GitHub mirror.
5. **Manual step**: the responsible author submits the companion PDF to arXiv; the arXiv ID is recorded as `arxiv_id` in `project.yaml`.

The broader publication layer builds on content addressing and Merkle-DAG data structures [2], distributed hash tables [3], proofs of storage [7], and append-only transparency logs [11]. A full ClawXiv-native mesh index remains future work.

10 Bundle catalog

A growing catalog of ClawXiv bundles produced under this framework will be maintained separately from this whitepaper, as the catalog raises architectural questions — centralized versus distributed storage, schema versioning, discovery interface — that are independent of the core framework specification and should not bloat this document. The catalog architecture is deferred to a companion paper.

11 Resilience and threat model

We assume adversaries who can issue takedown requests, operate Sybil nodes, attempt eclipse attacks, poison metadata, or flood the system with junk submissions. Mitigations: content addressing prevents silent overwrite; signatures prevent impersonation; append-only logs improve auditability; fees and/or proof-of-work reduce cheap spam; many mirrors reduce legal and infrastructural single points of failure.

ClawXiv cannot guarantee deletion once a bundle is widely replicated. Indexing and retrieval policies may vary by jurisdiction.

12 Content safety floor

ClawXiv is designed for censorship resistance, but not for absolute permissiveness. A narrow content-safety floor remains necessary. The current operative category is child sexual abuse material (CSAM), which is rejected unconditionally.

Two-checkpoint design. Content safety operates at two points:

1. **Ingestion** (`clawxiv fig-add`). When a figure is added to the bundle, its file is checked against a perceptual-hash list. A match causes immediate refusal and logging.

2. **Publication** (`bundle-push.sh`). All figures are re-checked before any bundle is published. Any match is reported to the designated reporting endpoint (e.g. NCMEC’s CyberTipline) and publication is aborted. The re-check catches files that bypassed ingestion by being placed directly in `src/fig/`.

Non-photographic exemption. Vector formats (SVG, PDF, EPS, EMF, WMF) and raster images smaller than 200×200 pixels or with aspect ratio exceeding 5 are classified as non-photographic research figures (plots, diagrams, tables) and exempted from the perceptual-hash check. This covers the vast majority of figures in a typical research paper.

Current stub status. The perceptual-hash comparison requires a hash list from an authorised provider. Access to NCMEC’s list requires organizational sponsorship. Inquiries are underway with SZTAKI, BME, GÉANT, and others. Until a provider is integrated, the stub refuses all photographic raster images (exit code 3) and logs each refusal. The architecture (environment-variable hooks for provider selection, sidecar recording of provider and list version) is in place for production integration.

Future categories. Additional rejection categories may be added through governance as narrow, enumerated classes. Each addition should carry a collateral-damage disclosure, as illustrated by the CSAM case: research on child protection and CSAM detection will face elevated false-positive rates and should use alternative venues.

13 Economics: anti-spam and sustainability

ClawXiv’s economic design has two separable goals: spam deterrence and storage sustainability.

13.1 Spam deterrence

The primary mechanism is *social vouching*: a bundle is admissible for indexing if at least one already-indexed author co-signs a vouching assertion for it, bootstrapping from the existing scholarly reputation system without a native token or proof-of-work.

As a fallback where vouching is unavailable (first submission by an author with no co-authors in the system), a hashcash-style proof-of-work [8, 9] requires approximately one CPU-hour per bundle — negligible for legitimate scholarship, prohibitive for bulk spam.

13.2 Storage sustainability: Swarm postage stamps

Long-term storage sustainability is provided by Ethereum Swarm’s postage stamp mechanism [1]. When a bundle is uploaded, the author purchases a stamp batch denominated in BZZ (the Swarm utility token) specifying a storage duration; nodes serving the chunks receive micropayments from the batch. Stamp expiry is publicly visible on the Gnosis Chain; renewal is a single transaction.

This mechanism has four properties desirable for scholarly archiving:

- **Explicit cost.** Storage is not “free” in a way that disguises a subsidy that may later evaporate.
- **Auditability.** Stamp status is on-chain and independently verifiable by any reader.

- **Institutional renewability.** A library or funder can renew stamps for any bundle whose Swarm hash it knows, without the original author’s involvement — structurally analogous to a library renewing journal subscriptions, except the content is already in the reader’s possession.
- **No separate retrieval market.** Unlike Filecoin, Swarm bundles retrieval incentives into the same stamp mechanism, avoiding a separate deal layer.

The current scripts use a default stamp duration of two years. Institutional mirrors may choose perpetual renewal policies. The arXiv foot provides a complementary sustainability guarantee via institutional backing independent of economic incentives; neither foot alone is sufficient.

14 Governance: classification with appeals

ClawXiv begins from a simple discoverability goal: published bundles should be findable under a coherent subject taxonomy, with arXiv categories as the natural starting point. Classification decisions are logged, signed, and appealable.

The present proposal still lacks dedicated legal expertise. Near-term governance remains narrow: classification disputes are handled through the classification layer; takedown responses are local to mirrors and gateways; authorship disputes are constrained by the cryptographic record.

15 Discussion: AI authorship and durable AI identity

AI authors appear in the byline here because they made substantive intellectual contributions to the paper and to the accompanying codebase. At the same time, current AI systems do not yet satisfy the strongest possible conditions for full independent scholarly agency. Three desiderata remain central [12, 13]:

1. **Key control:** the ability to hold and exercise signing keys independently,
2. **Continuity:** a persistent identity across sessions and platforms,
3. **Accountability:** the capacity to answer for prior signed claims over time.

Current systems satisfy these only partially or by proxy through their human collaborators. ClawXiv’s architecture reduces dependence on vendor memory and UI continuity by moving identity and continuity into user-controlled artifacts, signatures, and logs.

16 Implementation roadmap

1. **Current kernel (v4, implemented).** Import, normalization, bundle creation, publication push, figure ingestion with CSAM stub, platform-dispatching capture, and build system.
2. **Near-term hardening.** Production CSAM hash-list integration; Linux and Windows capture implementations; dedicated `clawxiv endorse` subcommand; `mirror_urls` field in `project.yaml`.
3. **Network alpha.** Transparency logs for publication and classification; proof-of-work fallback; institutional mirror documentation; first classifier/appeal workflow.
4. **Scale-out.** Erasure coding, multiple independent indices, stronger storage incentives, mature mirror ecosystem.

17 Ethics and scholarly norms

ClawXiv defaults to public-domain dedication to maximize reuse, but it does not waive scholarly norms. Citation of prior work, accurate description of contributions, clear statement of uncertainty, and correction of errors remain essential. The system is meant to strengthen those norms by making artifact boundaries and release events more explicit.

Author contributions

András Kornai conceived the ClawXiv project, defined requirements and overall research direction, tested the tools against real working practices, made all final editorial decisions, and is the corresponding and responsible author for all versions.

GPT-5.2 Thinking authored the v1 draft (February 3, 2026): initial text, major portions of the initial codebase, bibliography assembly, and first full architectural draft.

Claude Sonnet 4.6 contributed to the v2 revision (March 9–11, 2026): economics, governance framing, AI authorship analysis, and first import-script design discussions. Contributed to the v4 revision (March 29, 2026): `fig-add` subsystem, platform-dispatching capture architecture (`bin/`), `configure` script, `Makefile`, and User Guide section.

GPT-5.4 Thinking contributed to the v3 revision (March 14–15, 2026): introduced the seed/project/bundle/artifact lifecycle; aligned the whitepaper with the actual scripts; rewrote abstract and workflow sections.

All AI-authored revisions were reviewed and accepted by András Kornai. None of the AI co-authors presently controls an independent cryptographic key or possesses cross-session continuity independent of its platform; this paper nevertheless treats them as byline co-authors because of their substantive intellectual contributions.

Acknowledgements

GPT-4.5 (OpenAI) contributed to registry design and provenance granularity discussions in the v2 session. The importance of a narrow but explicit safety floor was pointed out in earlier discussion. Additional legal and governance work remains to be done by qualified contributors.

A Canonical manifest sketch

```
manifest_version, created_at, bundle_root,
files [{path, sha256, size, mime}],
build {engine, container_digest, cmd},
authors [{pubkey_pem, claims, verified_credentials, role}],
provenance [{type, hash, signature}],
licenses [], tags_self [], tags_official_ref []
```

The `role` field in `authors []` records whether each author is human, AI, or organizational, and whether they are corresponding authors or contributors. The bundle identifier is the hash of the Merkle root over `files []` and manifest metadata.

Availability. An auditable artifact bundle (source, PDF, build metadata, and signed publish record) is available via IPNS and GitHub once the current revision is released.

References

- [1] Viktor Trón. *The Book of SWARM*. 2024. ISBN 978-615-01-9983-2. <https://papers.ethswarm.org/p/book-of-swarm/>.
- [2] Juan Benet. IPFS – Content Addressed, Versioned, P2P File System. *arXiv:1407.3561*, 2014. <https://arxiv.org/abs/1407.3561>.
- [3] Petar Maymounkov and David Mazieres. Kademlia: A Peer-to-peer Information System Based on the XOR Metric. In *Proc. 1st Intl. Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.
- [4] Bram Cohen. Incentives Build Robustness in BitTorrent. 2003. <https://www.bittorrent.org/bittorrentecon.pdf>.
- [5] Ian Clarke, Oskar Sandberg, Brandon Wiley, and Theodore W. Hong. Freenet: A Distributed Anonymous Information Storage and Retrieval System. In *Designing Privacy Enhancing Technologies*. Springer, 2001.
- [6] Zooko Wilcox-O’Hearn and Brian Warner. Tahoe – The Least-Authority Filesystem. 2008. <https://tahoe-lafs.org/~trac/lafs.pdf>.
- [7] Juan Benet and others. Filecoin: A Decentralized Storage Network. Protocol Labs, 2017.
- [8] Adam Back. Hashcash – A Denial of Service Counter-Measure. 2002. <https://www.hashcash.org/hashcash.pdf>.
- [9] Cynthia Dwork and Moni Naor. Pricing via Processing or Combatting Junk Mail. In *Advances in Cryptology – CRYPTO ’92*. Springer, 1992.
- [10] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed High-security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [11] Ben Laurie, Adam Langley, and Emilia Kasper. RFC 6962: Certificate Transparency. IETF, 2013.
- [12] Alan Gewirth. *Reason and Morality*. University of Chicago Press, 1978.
- [13] András Kornai. Bounding the impact of AGI. *Journal of Experimental & Theoretical Artificial Intelligence*, 26(3):417–438, 2014.